

PROJECT REPORT

EMPLOYEE PAYROLL MANAGEMENT SYSTEM

A Java Desktop Application using Java Swing, JDBC and MySQL

Prepared by	Aarti Kumari
Project type	GitHub / Internship Portfolio
Development environment	NetBeans IDE
Database	MySQL

Note: Add your college/company name and project screenshots if required before final submission.

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope
6. Technologies Used
7. System Modules
8. System Workflow
9. Database Design
10. Setup and Installation
11. Testing
12. Security and Limitations
13. Future Enhancements
14. Conclusion
15. References

1. Abstract

The Employee Payroll Management System is a desktop application developed with Java Swing and connected to a MySQL database through JDBC. It is intended to organize employee information and provide a basic workflow for payroll processing and salary reporting.

The application includes login authentication, employee registration, employee details, payroll processing, and a salary report module. The project demonstrates the integration of a Java graphical user interface with relational database operations.

2. Introduction

Employee and salary information needs to be recorded consistently so that it can be reviewed and maintained. A desktop-based payroll system provides a single interface for entering employee details, processing salary values, and viewing salary records.

This project uses Java for application logic, Swing for the graphical user interface, JDBC for database connectivity, and MySQL for persistent storage. NetBeans IDE is used as the development environment.

3. Problem Statement

Maintaining employee details and salary records manually can lead to duplicated entries, inconsistent records, and difficulty retrieving information. The project addresses these basic record-management needs by bringing employee and payroll-related operations into one application connected to a database.

4. Objectives

- Provide a login screen for access to the application.
- Register employee details and store them in MySQL.
- View employee information through the Employee Details module.
- Support payroll processing using salary components such as basic salary, allowance, and deduction.
- Display salary records through the Salary Report module.
- Practice Java Swing, JDBC, SQL, and desktop application development.

5. Scope

The current scope covers a small desktop application for employee information and basic payroll records. It is suitable as a learning project and portfolio demonstration. It is not presented as a production-ready payroll platform and does not claim compliance with tax, statutory, or enterprise payroll rules.

6. Technologies Used

Java: Application logic and event handling.

Java Swing: Desktop windows, forms, buttons, and tables.

JDBC: Communication between Java and MySQL.

MySQL: Relational storage for employee and login records.

NetBeans IDE: Project development and GUI design.

7. System Modules

7.1 Login: Provides a login interface and checks the credentials configured for the application.

7.2 Employee Registration: Collects employee details such as employee ID, name, department, designation, phone, email, and salary-related fields, depending on the form configuration.

7.3 Employee Details: Provides a way to view stored employee information.

7.4 Payroll: Supports payroll-related entry or processing using basic salary, allowance, and deduction fields. The intended net salary calculation is $\text{basic salary} + \text{allowance} - \text{deduction}$; verify that the implementation follows this formula before relying on results.

7.5 Salary Report: Retrieves salary-related records from the employee table and displays them for review.

8. System Workflow

1. Launch the application from NetBeans.
2. Enter login credentials on the login screen.
3. Use the Home page to open Employee Registration, Employee Details, Payroll, or Salary Report.
4. Enter or review employee information as appropriate.
5. Save data to or retrieve data from the MySQL database through JDBC.
6. Open Salary Report to review the salary fields returned by the database.

9. Database Design

The project uses a MySQL database named **payroll**. The repository includes a starter SQL script at **database/payroll.sql** for creating the database and the employee and login tables.

Employee table: The starter schema includes employee ID, name, department, designation, phone, email, basic salary, allowance, deduction, and net salary fields.

Login table: The starter schema includes username and password fields. For real deployments, passwords should be stored using a secure password-hashing method rather than as plain text.

Important: Check the SQL script against the actual columns referenced by every Java form. If the application uses additional columns or tables, update the script before sharing the project.

10. Setup and Installation

1. Install a compatible JDK, NetBeans IDE, and MySQL Server.
2. Download or clone the GitHub repository to your computer.
3. Open the project in NetBeans using the project folder that contains **build.xml** and **nbproject**.
4. In MySQL, run the SQL statements in **database/payroll.sql** to create the starter database and tables.
5. Open **Connect.java** and configure the local database URL, username, and password. Do not commit real passwords to a public repository.
6. Ensure the MySQL Connector/J JDBC driver is available to the project classpath.
7. Clean and Build the project in NetBeans, then run the configured startup class (or the login/splash screen entry point).
8. If the app reports missing columns or tables, compare the SQL schema with the SQL statements in the Java source code.

11. Testing

The following manual checks are recommended before submission. Record the actual result for each test after running it; do not mark a test as passed unless it has been verified.

Test case	Expected result	Actual result
Login with valid credentials	Home page opens	To be tested
Register employee	Record is saved and retrievable	To be tested
Open Employee Details	Employee records are displayed	To be tested
Process payroll	Salary values are calculated/saved correctly	To be tested
Open Salary Report	Salary records are displayed	To be tested
Database unavailable	Clear error message is shown	To be tested

12. Security and Limitations

- Do not upload private configuration files, local machine paths, database passwords, or real employee personal data.
- If login credentials are hard-coded or stored in plain text, treat this as a learning/demo implementation only and improve it before real use.
- Validate numeric salary inputs and handle database errors clearly.
- This report describes the modules and starter schema; the exact behavior depends on the code committed to the repository.

13. Future Enhancements

- Secure password hashing and role-based access.
- Input validation and improved error handling.
- Export salary reports to PDF or Excel.
- Search, filtering, and date-wise payroll history.
- Automated unit and integration tests.
- Backup/restore support and audit logs.
- More complete payroll rules where required by the intended organization.

14. Conclusion

The Employee Payroll Management System demonstrates a basic Java Swing desktop application connected to MySQL through JDBC. Its modules provide a foundation for managing employee records, processing salary-related values, and viewing salary reports. Further validation, security improvements, and documentation of the exact database schema would make the project easier to maintain and reuse.

15. References

- Java documentation: <https://docs.oracle.com/en/java/>
- MySQL documentation: <https://dev.mysql.com/doc/>
- MySQL Connector/J documentation: <https://dev.mysql.com/doc/connector-j/en/>
- GitHub repository: https://github.com/Aartikumari635/Cognevance_Employee-Payroll-Management-System

